

Database Design Document (DDD)

For

Nexora: Multi-Vendor E-Commerce Marketplace

Version: 1.0.0-PROD

Date: July 26, 2026

Prepared By: Office of the Chief Database Architect & Enterprise Data Engineering Group

2. Document Information

Attribute	Details
Project Title	Nexora Multi-Vendor E-Commerce Marketplace
Document Type	Database Design Document (DDD)
Target Audience	Enterprise Architecture Board, Core Backend Engineering Teams, Database Administrators, QA Leads, Security Auditors
Security Classification	Proprietary & Confidential
Primary Technology Stack	MySQL 8, Java, Spring Boot, Spring Data JPA, Hibernate, Spring Security, JWT, Redis, Apache Kafka

3. Revision History

Date	Version	Author	Description / Change Summary
July 26, 2026	1.0.0	Chief Database Architect	Initial baseline database design document fully aligned with the Nexora SRS and System

			Architecture documents.
--	--	--	-------------------------

4. Table of Contents

1. Cover Page
2. Document Information
3. Revision History
4. Table of Contents
5. Database Overview
6. Database Naming Standards
7. Entity List
8. Entity Descriptions
9. Entity Relationships
10. ER Diagram
11. Detailed Table Design
12. Keys and Constraints
13. Indexing Strategy
14. Cascade Strategy
15. Fetch Strategy
16. Soft Delete Strategy
17. Audit Strategy
18. Transaction Strategy
19. Data Integrity Rules
20. Performance Optimization
21. Security Considerations
22. Backup and Recovery Strategy
23. Future Database Enhancements
24. Risks and Mitigation
25. Appendix

5. Database Overview

Purpose

This Database Design Document establishes the complete relational data schema for **Nexora**, a high-performance multi-vendor e-commerce marketplace. The design enforces structural integrity, referential consistency, transactional safety, and optimal query performance across all platform operations, including customer browsing, multi-vendor order fulfillment, inventory tracking, and administrative governance.

Database Choice

The platform utilizes **MySQL 8** as its primary relational data store. MySQL 8 provides robust ACID compliance, advanced indexing capabilities, native JSON support, window functions, and rigorous constraint enforcement necessary for financial ledger management and transactional ordering systems. Additionally, **Redis** is incorporated as a distributed caching and session state layer, while **Apache Kafka** manages asynchronous event distribution.

Design Philosophy

The database architecture strictly adheres to **Third Normal Form (3NF)** principles to eliminate data redundancy and anomalies. Foreign key constraints and referential integrity rules are enforced at the database engine level. Entities are structured to support a modular monolithic backend design with domain boundaries cleanly separated to facilitate future extraction into independent microservices.

Expected Growth

The system is architected to accommodate rapid expansion in data volume. Initial baseline projections anticipate 100,000 active customer accounts, 2,000 independent sellers, 500,000 active product SKUs, and up to 50,000 daily order transactions. Growth vectors account for high-frequency inventory updates and seasonal traffic spikes.

Scalability Considerations

To sustain high concurrency without performance degradation, the schema integrates optimized indexing strategies, strategic lazy loading, connection pooling (HikariCP), and future-proof design patterns such as read-replica separation paths, table partitioning hooks, and archiving readiness for historical transaction logs.

6. Database Naming Standards

- **Database:** Lowercase snake_case representation (nexora_production).
- **Tables:** Pluralized lowercase snake_case nouns (users, sellers, product_variants, orders).
- **Columns:** Singular lowercase snake_case descriptors (first_name, created_at, stock_quantity).
- **Primary Keys:** Standardized format utilizing id as a surrogate bigint identifier across all tables (id).
- **Foreign Keys:** Singular referenced table name followed by _id (user_id, category_id, seller_id).
- **Indexes:** Prefixed by index purpose and involved columns (idx_user_email, idx_order_status_created).
- **Unique Constraints:** Prefixed with uk_ followed by table and column descriptors (uk_users_email, uk_products_sku).
- **Check Constraints:** Prefixed with chk_ followed by constraint purpose (chk_product_price_positive).
- **Junction Tables:** Combination of both participating entity table names in alphabetical order (product_tag_mapping).
- **Views:** Prefixed with vw_ followed by descriptive view name (vw_seller_sales_summary).

7. Entity List

The Version 1 database schema comprises the following foundational entities, strictly aligned with the SRS and System Architecture specifications:

1. users
2. roles
3. user_roles
4. sellers
5. customer_profiles
6. addresses
7. categories
8. products
9. product_variants
10. product_images
11. inventories
12. carts
13. cart_items
14. wishlists
15. wishlist_items
16. orders
17. sub_orders
18. order_items
19. payments
20. shipments
21. reviews
22. notifications
23. coupons
24. audit_logs

8. Entity Descriptions

8.1 users

- **Purpose:** Stores core identity and authentication credentials for all platform users.
- **Responsibilities:** Managing authentication state, secure password hashes, email verification flags, and account status tracking.
- **Relationships:** One-to-One with customer_profiles, One-to-One with sellers, One-to-Many with addresses, Many-to-Many with roles.
- **Business Importance:** The foundational anchor for system security, access control, and user accountability.

8.2 roles

- **Purpose:** Defines distinct security roles recognized within the platform.
- **Responsibilities:** Storing role designations (ROLE_CUSTOMER, ROLE_SELLER,

ROLE_ADMIN).

- **Relationships:** Many-to-Many with users.
- **Business Importance:** Enforces role-based access control (RBAC) across system modules.

8.3 user_roles

- **Purpose:** Junction table resolving the Many-to-Many relationship between users and roles.
- **Responsibilities:** Associating user accounts with specific security permissions.
- **Relationships:** Many-to-One with users, Many-to-One with roles.
- **Business Importance:** Enables multi-role assignment capabilities where applicable.

8.4 sellers

- **Purpose:** Stores merchant-specific business profiles and verification data.
- **Responsibilities:** Tracking business registration details, tax identification numbers, corporate banking info, application status, and storefront customization properties.
- **Relationships:** One-to-One with users, One-to-Many with products, One-to-Many with sub_orders.
- **Business Importance:** Governs third-party merchant onboarding, compliance, and multi-vendor catalog management.

8.5 customer_profiles

- **Purpose:** Stores extended personal details for registered shoppers.
- **Responsibilities:** Managing shopper preferences, first and last names, phone numbers, and loyalty or profile attributes.
- **Relationships:** One-to-One with users, One-to-Many with orders, One-to-One with carts, One-to-One with wishlists.
- **Business Importance:** Provides personalized shopping experiences and order associations.

8.6 addresses

- **Purpose:** Stores physical billing and shipping locations for users.
- **Responsibilities:** Capturing street addresses, cities, state/provinces, postal codes, countries, and default address flags.
- **Relationships:** Many-to-One with users, One-to-Many with orders (shipping/billing).
- **Business Importance:** Critical for freight calculation, tax jurisdiction determination, and order fulfillment.

8.7 categories

- **Purpose:** Manages the hierarchical product taxonomy.
- **Responsibilities:** Structuring parent-child category relationships, category names, descriptions, and active status flags.

- **Relationships:** Self-referential Many-to-One (parent category), One-to-Many with products.
- **Business Importance:** Facilitates structured product discovery and faceted navigation.

8.8 products

- **Purpose:** Stores core product metadata managed by sellers.
- **Responsibilities:** Storing product titles, descriptions, brand details, category links, and base attributes.
- **Relationships:** Many-to-One with sellers, Many-to-One with categories, One-to-Many with product_variants, One-to-Many with product_images.
- **Business Importance:** The core merchant catalog offering available to customers.

8.9 product_variants

- **Purpose:** Represents distinct purchasable stock variations of a product (e.g., size, color).
- **Responsibilities:** Storing SKU codes, variant attribute key-values, retail prices, promotional discount prices, and weight/dimensions.
- **Relationships:** Many-to-One with products, One-to-One with inventories, One-to-Many with order_items, One-to-Many with cart_items.
- **Business Importance:** Enables granular pricing, SKU tracking, and inventory management per variant.

8.10 product_images

- **Purpose:** Stores media asset references associated with products.
- **Responsibilities:** Storing image URLs, display ordering indices, and primary image flags.
- **Relationships:** Many-to-One with products.
- **Business Importance:** Enhances customer visual engagement during product browsing.

8.11 inventories

- **Purpose:** Tracks real-time stock availability for individual product variants.
- **Responsibilities:** Maintaining current stock counts, reserved stock quantities, low-threshold alert levels, and optimistic locking version counters.
- **Relationships:** One-to-One with product_variants.
- **Business Importance:** Prevents over-selling and manages real-time warehouse stock synchronization.

8.12 carts

- **Purpose:** Maintains the active shopping session container for customers.
- **Responsibilities:** Associating shopping carts with customer profiles.
- **Relationships:** One-to-One with customer_profiles, One-to-Many with cart_items.
- **Business Importance:** Persistent storage of customer pre-checkout shopping selections.

8.13 cart_items

- **Purpose:** Stores individual variant line items within a customer cart.
- **Responsibilities:** Tracking selected quantities for specific product variants.
- **Relationships:** Many-to-One with carts, Many-to-One with product_variants.
- **Business Importance:** Manages line-item calculations prior to checkout conversion.

8.14 wishlists

- **Purpose:** Maintains customer saved product lists.
- **Responsibilities:** Associating customer profiles with saved items.
- **Relationships:** One-to-One with customer_profiles, One-to-Many with wishlist_items.
- **Business Importance:** Enhances customer retention and post-browse engagement.

8.15 wishlist_items

- **Purpose:** Stores individual product links within customer wishlists.
- **Responsibilities:** Associating wishlists with specific products.
- **Relationships:** Many-to-One with wishlists, Many-to-One with products.
- **Business Importance:** Enables quick transfer of saved items into shopping carts.

8.16 orders

- **Purpose:** Represents the top-level master order placed by a customer.
- **Responsibilities:** Storing overarching order metadata, total order amount, customer reference, shipping/billing address snapshots, and master payment status.
- **Relationships:** Many-to-One with customer_profiles, One-to-Many with sub_orders.
- **Business Importance:** Master financial and operational container for customer purchases.

8.17 sub_orders

- **Purpose:** Represents vendor-specific split orders derived from a master order.
- **Responsibilities:** Isolating order fulfillment per individual seller, tracking sub-order status (PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED), tracking shipment tracking numbers, and vendor payout ledgers.
- **Relationships:** Many-to-One with orders, Many-to-One with sellers, One-to-Many with order_items, One-to-One with shipments.
- **Business Importance:** Core multi-vendor architecture component separating fulfillment responsibilities across independent merchants.

8.18 order_items

- **Purpose:** Stores individual purchased line items within a sub-order.
- **Responsibilities:** Recording snapshot prices, purchased quantities, product variant references, and applicable seller commissions.
- **Relationships:** Many-to-One with sub_orders, Many-to-One with product_variants.

- **Business Importance:** Immutable record of purchased items, pricing, and revenue attribution.

8.19 payments

- **Purpose:** Records financial transaction logs associated with orders.
- **Responsibilities:** Storing payment gateway transaction IDs, payment methods, gateway response codes, transaction amounts, and payment status.
- **Relationships:** Many-to-One with orders.
- **Business Importance:** Ensures financial accountability, reconciliation, and refund auditing.

8.20 shipments

- **Purpose:** Tracks physical freight and courier logistics for sub-orders.
- **Responsibilities:** Storing courier partner names, tracking numbers, shipment milestones, and delivery timestamps.
- **Relationships:** One-to-One with sub_orders.
- **Business Importance:** Provides end-to-end delivery visibility for customers and administrators.

8.21 reviews

- **Purpose:** Captures customer feedback and ratings for purchased products.
- **Responsibilities:** Storing star ratings (1-5), textual review comments, verified purchase flags, seller replies, and moderation statuses.
- **Relationships:** Many-to-One with customer_profiles, Many-to-One with products, Many-to-One with sellers (seller replies).
- **Business Importance:** Drives marketplace trust, product quality assurance, and merchant reputation.

8.22 notifications

- **Purpose:** Manages system-generated alerts and communications.
- **Responsibilities:** Storing notification payloads, recipient references, delivery channels (Email, SMS, In-App), and read status flags.
- **Relationships:** Many-to-One with users.
- **Business Importance:** Ensures timely user communication regarding orders, account events, and system updates.

8.23 coupons

- **Purpose:** Manages promotional discount codes and campaigns.
- **Responsibilities:** Storing discount codes, discount percentage/fixed values, expiration dates, usage limits, and active status flags.
- **Relationships:** One-to-Many with orders.
- **Business Importance:** Drives promotional marketing campaigns and sales conversion

incentives.

8.24 audit_logs

- **Purpose:** Maintains immutable audit trails of sensitive system and administrative actions.
- **Responsibilities:** Recording actor IDs, action types, target entities, IP addresses, timestamps, and change payloads.
- **Relationships:** Many-to-One with users.
- **Business Importance:** Regulatory compliance, security forensic analysis, and administrative accountability.

9. Entity Relationships

- **One-to-One Relationships:**
 - users to customer_profiles: Every registered customer account has exactly one detailed profile.
 - users to sellers: An approved seller account corresponds to exactly one base user identity.
 - customer_profiles to carts: A customer maintains exactly one active shopping cart container.
 - customer_profiles to wishlists: A customer maintains exactly one personal wishlist container.
 - product_variants to inventories: Each individual product variant maintains a dedicated inventory stock record.
 - sub_orders to shipments: Each vendor sub-order is associated with a specific shipment tracking record.
- **One-to-Many Relationships:**
 - users to addresses: A user can configure multiple shipping and billing addresses.
 - categories to categories (Self-referential): A category can have multiple sub-categories within the hierarchical tree.
 - sellers to products: A registered seller can list multiple products.
 - products to product_variants: A base product can have multiple distinct variants (sizes, colors).
 - products to product_images: A product can feature multiple high-resolution image assets.
 - carts to cart_items: A cart contains multiple line-item selections.
 - wishlists to wishlist_items: A wishlist contains multiple saved product references.
 - customer_profiles to orders: A customer can place multiple master orders over time.
 - orders to sub_orders: A master order is split into one or more vendor-specific sub-orders during checkout.
 - sellers to sub_orders: A seller fulfills multiple sub-orders assigned to their storefront.
 - sub_orders to order_items: A sub-order contains multiple purchased product variant line items.
 - users to audit_logs: A user (particularly administrators) generates audit log entries across actions.

- users to notifications: A user receives multiple system notifications.
- orders to payments: An order may have multiple payment attempts or refund transactions.
- customers / products to reviews: A product receives multiple customer reviews.
- **Many-to-Many Relationships:**
 - users to roles (Resolved via user_roles junction table): Users can possess multiple security roles, and roles can be assigned to multiple users.

10. ER Diagram



11. Detailed Table Design

11.1 users

- **Purpose:** Core user credentials and status tracking.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - email: VARCHAR(255) (Unique, Not Null)
 - password_hash: VARCHAR(255) (Not Null)
 - is_email_verified: BOOLEAN (Default: FALSE, Not Null)
 - account_status: VARCHAR(50) (Default: 'ACTIVE', Not Null)
 - failed_login_attempts: INT (Default: 0, Not Null)
 - lockout_until: TIMESTAMP (Nullable)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)
 - deleted_at: TIMESTAMP (Nullable - Soft Delete)

11.2 roles

- **Purpose:** System security role definitions.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - name: VARCHAR(50) (Unique, Not Null - e.g., ROLE_CUSTOMER, ROLE_SELLER, ROLE_ADMIN)
 - description: VARCHAR(255) (Nullable)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.3 user_roles

- **Purpose:** Junction table linking users to security roles.
- **Columns:**
 - user_id: BIGINT (Foreign Key to users.id, Not Null)
 - role_id: BIGINT (Foreign Key to roles.id, Not Null)
 - *Composite Primary Key:* (user_id, role_id)

11.4 sellers

- **Purpose:** Merchant business profiles and verification data.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - user_id: BIGINT (Foreign Key to users.id, Unique, Not Null)
 - store_name: VARCHAR(150) (Unique, Not Null)
 - store_description: TEXT (Nullable)

- tax_identification_number: VARCHAR(50) (Unique, Not Null)
- bank_account_details: VARCHAR(255) (Not Null - Encrypted)
- application_status: VARCHAR(50) (Default: 'PENDING', Not Null)
- rejection_reason: TEXT (Nullable)
- commission_rate: DECIMAL(5,2) (Default: 10.00, Not Null)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
- updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.5 customer_profiles

- **Purpose:** Customer personal details and preferences.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - user_id: BIGINT (Foreign Key to users.id, Unique, Not Null)
 - first_name: VARCHAR(100) (Not Null)
 - last_name: VARCHAR(100) (Not Null)
 - phone_number: VARCHAR(20) (Nullable)
 - avatar_url: VARCHAR(512) (Nullable)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.6 addresses

- **Purpose:** Customer and shipping physical addresses.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - user_id: BIGINT (Foreign Key to users.id, Not Null)
 - street_address: VARCHAR(255) (Not Null)
 - city: VARCHAR(100) (Not Null)
 - state_province: VARCHAR(100) (Not Null)
 - postal_code: VARCHAR(20) (Not Null)
 - country: VARCHAR(100) (Not Null)
 - is_default: BOOLEAN (Default: FALSE, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.7 categories

- **Purpose:** Hierarchical category taxonomy.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - parent_id: BIGINT (Foreign Key to categories.id, Nullable)

- name: VARCHAR(100) (Not Null)
- slug: VARCHAR(120) (Unique, Not Null)
- description: TEXT (Nullable)
- is_active: BOOLEAN (Default: TRUE, Not Null)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
- updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.8 products

- **Purpose:** Master product catalog listings.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - seller_id: BIGINT (Foreign Key to sellers.id, Not Null)
 - category_id: BIGINT (Foreign Key to categories.id, Not Null)
 - title: VARCHAR(255) (Not Null)
 - slug: VARCHAR(255) (Unique, Not Null)
 - description: TEXT (Not Null)
 - brand: VARCHAR(100) (Nullable)
 - is_active: BOOLEAN (Default: TRUE, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)
 - deleted_at: TIMESTAMP (Nullable - Soft Delete)

11.9 product_variants

- **Purpose:** Distinct stock variations and pricing for products.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - product_id: BIGINT (Foreign Key to products.id, Not Null)
 - sku: VARCHAR(100) (Unique, Not Null)
 - attributes_json: JSON (Not Null - e.g., {"Size": "XL", "Color": "Blue"})
 - price: DECIMAL(10,2) (Not Null)
 - discount_price: DECIMAL(10,2) (Nullable)
 - weight_kg: DECIMAL(8,2) (Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)
 - deleted_at: TIMESTAMP (Nullable - Soft Delete)

11.10 product_images

- **Purpose:** Product media asset references.
- **Columns:**

- id: BIGINT (Primary Key, Auto Increment)
- product_id: BIGINT (Foreign Key to products.id, Not Null)
- image_url: VARCHAR(512) (Not Null)
- display_order: INT (Default: 0, Not Null)
- is_primary: BOOLEAN (Default: FALSE, Not Null)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.11 inventories

- **Purpose:** Real-time stock quantity tracking per variant.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - product_variant_id: BIGINT (Foreign Key to product_variants.id, Unique, Not Null)
 - stock_quantity: INT (Default: 0, Not Null)
 - reserved_quantity: INT (Default: 0, Not Null)
 - low_stock_threshold: INT (Default: 5, Not Null)
 - version: BIGINT (Default: 0, Not Null - Optimistic Locking)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.12 carts

- **Purpose:** Customer shopping cart session container.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - customer_profile_id: BIGINT (Foreign Key to customer_profiles.id, Unique, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.13 cart_items

- **Purpose:** Line items within customer shopping carts.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - cart_id: BIGINT (Foreign Key to carts.id, Not Null)
 - product_variant_id: BIGINT (Foreign Key to product_variants.id, Not Null)
 - quantity: INT (Default: 1, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.14 wishlists

- **Purpose:** Customer saved product list container.
- **Columns:**

- id: BIGINT (Primary Key, Auto Increment)
- customer_profile_id: BIGINT (Foreign Key to customer_profiles.id, Unique, Not Null)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
- updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.15 wishlist_items

- **Purpose:** Saved product links in customer wishlists.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - wishlist_id: BIGINT (Foreign Key to wishlists.id, Not Null)
 - product_id: BIGINT (Foreign Key to products.id, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.16 orders

- **Purpose:** Master purchase order container.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - customer_profile_id: BIGINT (Foreign Key to customer_profiles.id, Not Null)
 - total_amount: DECIMAL(10,2) (Not Null)
 - shipping_address_snapshot: TEXT (Not Null)
 - billing_address_snapshot: TEXT (Not Null)
 - payment_status: VARCHAR(50) (Default: 'PENDING', Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.17 sub_orders

- **Purpose:** Vendor-specific split fulfillment orders.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - order_id: BIGINT (Foreign Key to orders.id, Not Null)
 - seller_id: BIGINT (Foreign Key to sellers.id, Not Null)
 - sub_total: DECIMAL(10,2) (Not Null)
 - shipping_fee: DECIMAL(10,2) (Not Null)
 - commission_amount: DECIMAL(10,2) (Not Null)
 - fulfillment_status: VARCHAR(50) (Default: 'PENDING', Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.18 order_items

- **Purpose:** Purchased line items within sub-orders.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - sub_order_id: BIGINT (Foreign Key to sub_orders.id, Not Null)
 - product_variant_id: BIGINT (Foreign Key to product_variants.id, Not Null)
 - quantity: INT (Not Null)
 - unit_price: DECIMAL(10,2) (Not Null)
 - total_price: DECIMAL(10,2) (Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.19 payments

- **Purpose:** Financial transaction ledger records.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - order_id: BIGINT (Foreign Key to orders.id, Not Null)
 - transaction_reference: VARCHAR(150) (Unique, Not Null)
 - payment_gateway: VARCHAR(50) (Not Null)
 - amount: DECIMAL(10,2) (Not Null)
 - status: VARCHAR(50) (Default: 'PENDING', Not Null)
 - gateway_response_json: JSON (Nullable)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.20 shipments

- **Purpose:** Courier tracking and fulfillment logistics.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - sub_order_id: BIGINT (Foreign Key to sub_orders.id, Unique, Not Null)
 - courier_partner: VARCHAR(100) (Not Null)
 - tracking_number: VARCHAR(150) (Unique, Not Null)
 - shipment_status: VARCHAR(50) (Default: 'PREPARING', Not Null)
 - shipped_at: TIMESTAMP (Nullable)
 - delivered_at: TIMESTAMP (Nullable)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
 - updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.21 reviews

- **Purpose:** Customer product ratings and feedback.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)

- customer_profile_id: BIGINT (Foreign Key to customer_profiles.id, Not Null)
- product_id: BIGINT (Foreign Key to products.id, Not Null)
- rating: INT (Not Null - Check constraint: 1 to 5)
- comment: TEXT (Nullable)
- is_verified_purchase: BOOLEAN (Default: TRUE, Not Null)
- seller_reply: TEXT (Nullable)
- moderation_status: VARCHAR(50) (Default: 'APPROVED', Not Null)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)
- updated_at: TIMESTAMP (Default: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, Not Null)

11.22 notifications

- **Purpose:** Multi-channel user notifications.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - user_id: BIGINT (Foreign Key to users.id, Not Null)
 - title: VARCHAR(150) (Not Null)
 - message: TEXT (Not Null)
 - channel: VARCHAR(50) (Not Null - e.g., EMAIL, SMS, IN_APP)
 - is_read: BOOLEAN (Default: FALSE, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.23 coupons

- **Purpose:** Promotional discount codes.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - code: VARCHAR(50) (Unique, Not Null)
 - discount_type: VARCHAR(20) (Not Null - PERCENTAGE or FIXED)
 - discount_value: DECIMAL(10,2) (Not Null)
 - min_order_amount: DECIMAL(10,2) (Default: 0.00, Not Null)
 - expiration_date: TIMESTAMP (Not Null)
 - usage_limit: INT (Not Null)
 - times_used: INT (Default: 0, Not Null)
 - is_active: BOOLEAN (Default: TRUE, Not Null)
 - created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

11.24 audit_logs

- **Purpose:** Immutable administrative and security audit trail.
- **Columns:**
 - id: BIGINT (Primary Key, Auto Increment)
 - user_id: BIGINT (Foreign Key to users.id, Nullable)
 - action: VARCHAR(100) (Not Null)

- target_entity: VARCHAR(100) (Not Null)
- target_id: BIGINT (Nullable)
- ip_address: VARCHAR(45) (Not Null)
- payload_json: JSON (Nullable)
- created_at: TIMESTAMP (Default: CURRENT_TIMESTAMP, Not Null)

12. Keys and Constraints

- **Primary Keys:** Every table is mandated to possess a surrogate primary key column named id defined as BIGINT AUTO_INCREMENT. Junction tables utilize composite primary keys combining foreign key identifiers.
- **Foreign Key Constraints:** Foreign keys enforce referential integrity across related tables with strict cascading rules (RESTRICT or CASCADE where appropriate) to prevent orphaned records.
- **Unique Constraints:** Applied strictly to business-critical unique attributes such as users.email, sellers.store_name, sellers.tax_identification_number, categories.slug, products.slug, product_variants.sku, and payments.transaction_reference.
- **Composite Keys:** Enforced on junction tables (user_roles linking user_id and role_id).
- **Check Constraints:** Enforced on numeric and logical bounds (e.g., product prices must be greater than zero, review ratings must fall between 1 and 5 inclusive, stock quantities cannot be negative).

13. Indexing Strategy

- **Search Indexes:** Full-text indexes on products.title and products.description to enable lightning-fast catalog search queries.
- **Filtering Indexes:** B-Tree indexes on products.category_id, product_variants.price, products.brand, and sub_orders.fulfillment_status to accelerate faceted filtering.
- **Sorting Indexes:** Composite indexes on products (created_at DESC) and product_variants (price ASC) to optimize default catalog sorting performance.
- **Authentication Indexes:** Unique index on users.email for sub-millisecond credential lookup during authentication.
- **Order Processing Indexes:** Indexes on orders.customer_profile_id and sub_orders.seller_id to streamline order history retrieval and vendor dashboard rendering.
- **Reporting Indexes:** Composite indexes on sub_orders (seller_id, created_at) to accelerate sales analytics aggregation and financial reporting.

14. Cascade Strategy

- **Cascade Persist:** Enabled on master-detail relationships (e.g., persisting a Product automatically persists associated ProductVariant and ProductImage entities).
- **Cascade Merge:** Enabled for container relationships where updates to parent aggregates cascade to child elements.
- **Cascade Remove:** Restricted strictly to owned child entities where independent lifecycle existence is impossible (e.g., deleting a Cart cascades to delete associated CartItem records). **Never** applied to financial records, orders, or user accounts.

- **Cascade Refresh:** Applied across transactional entities to ensure object state synchronization with database triggers and defaults.
- **Where Cascading Should NOT Be Used:** Cascading remove is strictly prohibited on users, sellers, products, and orders to prevent accidental cascading deletion of critical business assets or financial ledgers. Soft deletes are used instead.

15. Fetch Strategy

- **Lazy Loading (FetchType.LAZY):** Applied as the default fetching strategy for all @OneToMany, @ManyToOne, and @ManyToMany associations. This prevents unrequested data loading and avoids performance bottlenecks (N+1 query anti-patterns).
- **Eager Loading (FetchType.EAGER):** Strictly restricted and avoided across relationships. Where entity graphs require associated data in a single round-trip, explicit query joins (JOIN FETCH) or repository entity graphs are utilized on a per-query basis.

16. Soft Delete Strategy

- **Identified Tables:** users, products, and product_variants support soft deletes via a nullable deleted_at timestamp column.
- **Rationale:** Soft deletion preserves historical relational integrity for completed orders, financial invoices, and analytics audit trails while hiding deleted items from active catalog browsing and customer purchasing workflows.

17. Audit Strategy

- **Audit Fields:** Major transactional and entity tables incorporate standardized audit columns:
 - created_at: TIMESTAMP recording creation time.
 - updated_at: TIMESTAMP recording last modification time.
 - deleted_at: TIMESTAMP recording soft deletion time (where applicable).
- **Database Triggers & ORM Auditing:** Automatic population of audit timestamps is managed via database default timestamps and JPA entity listeners (@EntityListeners(AuditingEntityListener.class)).

18. Transaction Strategy

- **Order Placement:** Executed within a strict ACID transactional boundary (@Transactional(isolation = Isolation.REPEATABLE_READ)) ensuring atomic cart conversion, order generation, sub-order creation, inventory reservation, and payment initiation.
- **Payment Processing:** Managed under isolated transactions with idempotency checks to prevent duplicate ledger postings upon asynchronous webhook receipt.
- **Inventory Update:** Leverages pessimistic or optimistic locking (@Version) within independent transactional boundaries to prevent race conditions and inventory over-allocation.
- **Seller Approval:** Admin approval transactions update seller status, provision system roles, and trigger Kafka event publishing atomically.
- **Product Management:** Catalog creation and variant modifications execute within

transactional boundaries ensuring referential integrity across variant attributes and image associations.

19. Data Integrity Rules

1. User email addresses must be globally unique across the platform.
2. A seller store name must be globally unique.
3. A seller tax identification number must be unique.
4. User passwords must be stored strictly as secure bcrypt hashes.
5. Product SKUs must be globally unique.
6. Product base prices must be greater than zero.
7. Promotional discount prices cannot exceed original product prices.
8. Product inventory stock quantities cannot be negative.
9. A shopping cart is strictly bound to one customer profile.
10. A wishlist is strictly bound to one customer profile.
11. An order must contain at least one sub-order and associated order item.
12. A sub-order must reference a valid existing master order and an approved seller.
13. Sub-order fulfillment status values are strictly restricted to defined lifecycle states.
14. Payment transaction references must be unique.
15. Payment amounts must match master order total amounts.
16. A shipment record must reference exactly one valid sub-order.
17. Courier tracking numbers must be unique across active shipments.
18. Customer review ratings must be integer values between 1 and 5 inclusive.
19. A customer can review a purchased product variant only once per delivered order.
20. Verified purchase review flags require a confirmed delivered order status.
21. Coupon codes must be globally unique.
22. Coupon expiration dates must be set in the future upon creation.
23. Audit log records are immutable and cannot be updated or deleted.
24. Category slugs must be unique across the category tree.
25. A child category cannot reference itself as a parent category.
26. Low stock thresholds cannot be set to negative values.
27. Commission rates must be expressed as valid percentage decimals between 0.00 and 100.00.
28. Soft-deleted users cannot authenticate against the platform.
29. Soft-deleted products cannot be added to customer shopping carts.
30. Foreign key constraints reject deletion of parent records that possess active dependent children.
31. Failed login attempts increment an account counter up to a maximum limit before triggering temporary logout.
32. Address records must belong to an existing valid user account.
33. Cart items must reference existing active product variants.
34. Wishlist items must reference existing active products.
35. Order item prices are snapshotted at checkout and remain immutable regardless of subsequent product price changes.

36. Seller bank account details must be encrypted at rest.
37. Default address flags can only be active for one address per user per address type.
38. Notification records must link to a valid registered user.
39. Image display order indices must be non-negative integers.
40. Primary product image flags can be true for at most one image per product.
41. Sub-order commission amounts must be accurately computed based on the assigned seller commission rate.
42. Refund transactions cannot exceed the original payment transaction amount.
43. Database schema migrations must execute in strict chronological sequence.
44. Timestamp columns default to current UTC timestamps to maintain cross-region consistency.
45. Database connection pooling parameters must enforce strict maximum connection limits.
46. Database user privileges must follow the principle of least privilege.
47. Sensitive query parameters must utilize prepared statements to prevent injection vulnerabilities.
48. Unused database connections must be reclaimed after timeout thresholds.
49. Database deadlock retry mechanisms must be handled gracefully at the service transaction boundary.
50. Index fragmentation must be monitored and mitigated through regular maintenance scheduling.

20. Performance Optimization

- **Partitioning Strategy (Future):** Future enhancements will implement range partitioning on audit_logs and orders by timestamp intervals to maintain rapid query performance as historical data volumes expand.
- **Archiving Strategy:** Completed orders and audit logs older than 3 years will be archived into cold storage historical tables to maintain lean operational database schemas.
- **Index Optimization:** Regular analysis of slow query execution plans will guide index additions and eliminate redundant indexing overhead.
- **Query Optimization:** Elimination of N+1 select problems via targeted JPQL fetch joins and projection queries for read-heavy dashboard operations.
- **Connection Pooling:** Integration of HikariCP connection pooling configured with optimal pool sizing, connection timeout limits, and idle connection reclamation.

21. Security Considerations

- **Sensitive Data:** Personal Identifiable Information (PII) such as phone numbers and physical addresses are protected via application-level encryption where mandated by compliance standards.
- **Password Storage:** Passwords are stored exclusively as bcrypt cryptographic hashes with a work factor of 12; plaintext passwords are never persisted.
- **PII Protection:** Customer and seller financial details (bank account strings) are encrypted at rest within database storage.

- **Encryption Recommendations:** Transport Layer Security (TLS 1.3) is enforced for all database client-server connections. Database storage volumes utilize transparent data encryption (TDE) at the infrastructure tier.

22. Backup and Recovery Strategy

- **Backup Frequency:** Automated incremental database backups are executed every 6 hours, supplemented by full daily backups retained in geographically redundant cloud object storage for 30 days.
- **Recovery Strategy:** Point-in-Time Recovery (PITR) procedures are supported via MySQL binary logging (binlogs) to enable granular data restoration in the event of corruption.
- **Disaster Recovery Considerations:** Cross-region database replication ensures rapid failover capabilities, targeting a Recovery Time Objective (RTO) of under 1 hour and a Recovery Point Objective (RPO) of under 5 minutes.

23. Future Database Enhancements

1. Implementation of read-replicas to scale read-heavy catalog query performance independently from write operations.
2. Adoption of database sharding strategies to horizontally partition user and order tables across multiple database instances.
3. Implementation of multi-tenant database schema isolation strategies for enterprise-grade data privacy.
4. Integration of Elasticsearch clusters for advanced full-text product search and faceted filtering.
5. Adoption of historical data warehousing schemas (Star Schema) for advanced business intelligence analytics.
6. Implementation of event-sourcing data persistence models for immutable financial ledgers.
7. Integration of automated database partitioning policies for high-volume telemetry and log tables.
8. Adoption of multi-region active-active database replication topologies.
9. Implementation of automated database indexing tuning based on query execution statistics.
10. Integration of decentralized supplier dropshipping database mapping entities.
11. Adoption of columnar database storage formats for high-speed analytical reporting queries.
12. Implementation of subscription-based recurring billing and automated renewal entities.
13. Integration of customer loyalty reward point ledger tables and tier management attributes.
14. Adoption of distributed transaction coordinators (Saga pattern persistence) upon microservice migration.
15. Implementation of automated database schema change auditing via version-controlled migration pipelines.
16. Integration of multi-currency exchange rate mapping tables and localization schemas.

17. Adoption of cryptographic tokenization for stored payment instruments.
18. Implementation of automated data masking routines for non-production staging environments.
19. Integration of real-time streaming data connectors (Debezium / CDC) for change data capture into Kafka.
20. Adoption of advanced database compression algorithms to reduce storage footprint on historical tables.
21. Implementation of fine-grained row-level security policies for multi-vendor data isolation.
22. Integration of automated database health diagnostics and deadlock detection monitors.
23. Adoption of cloud-native serverless database scaling configurations.
24. Implementation of digital asset metadata management tables for augmented reality product models.
25. Integration of wholesale B2B bulk quotation and tier-pricing relational tables.
26. Adoption of advanced spatial indexing for location-based warehouse shipping routing.
27. Implementation of automated data retention purging jobs for expired guest shopping carts.
28. Integration of sentiment analysis storage attributes for customer product reviews.
29. Adoption of zero-downtime schema migration testing frameworks in CI/CD pipelines.
30. Establishment of comprehensive disaster recovery simulation drills on secondary database clusters.

24. Risks and Mitigation

Risk Identified	Impact	Probability	Mitigation Strategy
High Concurrency Inventory Deadlocks	High	Medium	Implement optimistic locking (@Version) and Redis distributed locks for stock decrements.
Database Connection Pool Exhaustion	High	Medium	Configure HikariCP connection pooling with strict maximum pool limits and timeout thresholds.
Unoptimized	Medium	High	Enforce strict code

Query Performance Degradation			review query profiling, execution plan analysis, and mandatory indexing rules.
Data Corruption During System Failure	High	Low	Enforce strict ACID transaction boundaries, foreign key constraints, and automated binlog replication.
Uncontrolled Table Growth (Audit/Orders)	Medium	High	Implement table partitioning and archival strategies for historical data retention.

25. Appendix

Appendix A: Reference Standards

- Third Normal Form (3NF) Database Normalization Principles.
- MySQL 8 Reference Manual and Performance Tuning Guidelines.
- OWASP Database Security Guidance and SQL Injection Prevention Standards.

Appendix B: Database Design Sign-Off

- **Chief Database Architect:** _____ Date: _____
- **Lead Backend Engineer:** _____ Date: _____
- **Database Administrator:** _____ Date: _____